

Graph-Based State Estimation, Fault Diagnosis, and Measurement Security on a Distribution Feeder

1 abstract

This report presents a self-study project on distribution system monitoring using the IEEE 33-bus radial feeder. A classical weighted-least-squares (WLS) state estimator is implemented and compared against a GraphSAGE graph neural network trained on the same measurement data. Fault diagnosis is based on symmetrical-component theory, and a false-data-injection (FDI) detector is built and tested. Results are reported as obtained, including cases where the method does not perform well, along with an explanation of why.

2 Introduction

The goal of this project was to combine three related topics in power systems into a single pipeline: state estimation, fault diagnosis, and cybersecurity for smart-grid measurements. A single graph neural network is trained to estimate the system state from partial measurements, and this same model is then reused for two tasks: classifying fault type, and detecting spoofed (falsified) measurements.

An earlier version of this project had two problems: fault labels were generated using arbitrary numbers instead of real physics, and the attack detector had a design flaw that let it “see” the value it was supposed to check. This report describes a corrected version that fixes both issues and reports the results honestly, including a new limitation that only became visible after the first bug was fixed.

3 Background

3.1 Power Flow

Each bus i in the network has a voltage V_i and a power injection $S_i = P_i + jQ_i$, related through the bus admittance matrix Y_{bus} by

$$S_i = V_i \left(\sum_k Y_{ik} V_k \right)^* . \quad (1)$$

`pandapower`’s power-flow solver is used as ground truth. Y_{bus} is also built by hand in this project, both to understand it and because it is needed directly by the WLS estimator.

3.2 State Estimation (WLS)

State estimation is the reverse of power flow: given noisy, partial measurements z , estimate the true state x . The measurement model is

$$z = h(x) + e, \quad e \sim \mathcal{N}(0, R), \quad (2)$$

and the Weighted Least Squares estimate minimizes

$$\hat{x} = \arg \min_x (z - h(x))^T R^{-1} (z - h(x)). \quad (3)$$

Since h is nonlinear, this is solved iteratively (Gauss–Newton), linearizing with the Jacobian $H = \partial h / \partial x$ at each step. The residual at convergence, $r = z - h(\hat{x})$, is also the standard tool for bad-data detection: a normalized residual $|r_i| / \sigma_i$ that is unusually large flags a suspect measurement. This idea is reused later for attack detection.

3.3 GraphSAGE

A distribution feeder is a graph: buses are nodes, lines are edges. GraphSAGE updates each node using its own features and the average of its neighbors’ features:

$$h_v^{(\ell+1)} = \text{ReLU} \left(W_{\text{self}} h_v^{(\ell)} + W_{\text{neigh}} \cdot \frac{1}{|N(v)|} \sum_{u \in N(v)} h_u^{(\ell)} \right). \quad (4)$$

Two layers are stacked, followed by a linear layer that outputs the predicted state $[V_m, V_a]$ at each bus. This is implemented directly in PyTorch, without a graph library, since the topology is small and fixed.

3.4 Fault Types and Symmetrical Components

Real faults are usually unbalanced across the three phases. Fortescue’s transform splits phase quantities into positive, negative, and zero sequence components. Each fault type connects these three sequence networks differently, giving different fault currents (Table 1). The important point: only the balanced three-phase fault has zero negative-sequence current. Every unbalanced fault has a nonzero negative-sequence signature, which is a standard, physically real diagnostic quantity, and is the basis for the fault features used in this project.

Table 1: Fault current formulas from symmetrical components.

Fault type	Positive-sequence current	Negative-sequence current
Three-phase (LLL)	$E / (Z_1 + Z_f)$	0
Line-to-line (LL)	$E / (Z_1 + Z_2 + Z_f)$	$-I_{a1}$
Single line-ground (LG)	$E / (Z_1 + Z_2 + Z_0 + 3Z_f)$	I_{a1}
Double line-ground (LLG)	$E / (Z_1 + \frac{Z_2(Z_0 + 3Z_f)}{Z_2 + Z_0 + 3Z_f})$	current-divider split

3.5 False Data Injection

A false-data-injection (FDI) attack adds a chosen corruption to the measurements. Liu, Ning, and Reiter showed that for the linear model $z = Hx$, an attack $a = Hc$ shifts the estimate without changing the residual at all – it is invisible to the classical bad-data check. Building an equally stealthy attack against a nonlinear or learned model is harder. This project uses a simpler attack: a random bias added to some meters, tested against an estimator that is not allowed to see that meter’s own reading.

4 Methodology

4.1 Feeder Model

The IEEE 33-bus radial feeder was loaded using `pandapower`’s `case33bw()`. The standard test system has 32 normally-closed lines and 5 normally-open tie lines used for reconfiguration studies (37 lines total). Only in-service lines were used to build the network graph; using all 37 lines by mistake would add 5 extra edges that do not carry current, and this was checked and confirmed by verifying the resulting graph is a tree.

A per-unit admittance matrix Y_{bus} was built directly from each line’s resistance and reactance. Before using this model for anything else, it was checked against `pandapower`’s own solution: the true solved voltages were passed back through Eq. (1), and the resulting power injections were compared against `pandapower`’s own values.

4.2 Measurement Data

3,000 snapshots were generated by randomly scaling every load and re-solving power flow. Each snapshot has a noisy, partial measurement set: power injections at every bus, and voltage magnitude at a random 30% of buses only. An 80/20 train/test split (2,400 / 600 snapshots) was fixed and reused for every comparison.

4.3 Classical WLS Estimator

The Gauss–Newton method was implemented in NumPy. Since $h(x)$ is a simple matrix-vector product using the hand-built Y_{bus} , its Jacobian was computed by finite differences without needing repeated calls to `pandapower`. Iteration started from a flat voltage profile and ran for up to 10 steps.

4.4 GraphSAGE Estimator

A two-layer GraphSAGE model (hidden size 32) was trained with Adam for 400 epochs, using mean-squared error against the true state at every bus.

4.5 Fault Features

Since `pandapower`’s standard solver only handles balanced faults, fault features were computed directly from the formulas in Table 1. For a radial (tree) network, the impedance from the source to any bus – and the shared impedance between any two buses – can be found just by looking at how much of their paths back to the source overlap, without needing a full impedance-matrix inversion. A fixed pre-fault voltage of 1.0 p.u. was assumed, $Z_2 = Z_1$, $Z_0 = 2.5Z_1$, and the fault impedance Z_f was drawn randomly for every sample. Five summary features (minimum, mean,

and standard deviation of the voltage profile, plus two negative-sequence statistics) were used to train a Random Forest classifier.

4.6 FDI Attack and Detection

An attacker was simulated by adding a random bias to 15% of the metered voltage readings. The earlier version of this project let the detector see the very meter reading it was checking, which made the test meaningless. The fix: a second model is trained with all voltage-meter inputs set to zero, for every bus, always – so it can never see a direct voltage reading and must infer voltage purely from power injections and network structure. Its prediction is compared against the reported (possibly attacked) reading to build a residual, tested both directly and with a Random Forest classifier.

5 Results

5.1 Network Topology

After filtering to in-service lines, exactly 32 lines connect the 33 buses, and the result is confirmed to be a tree (Figure 1).

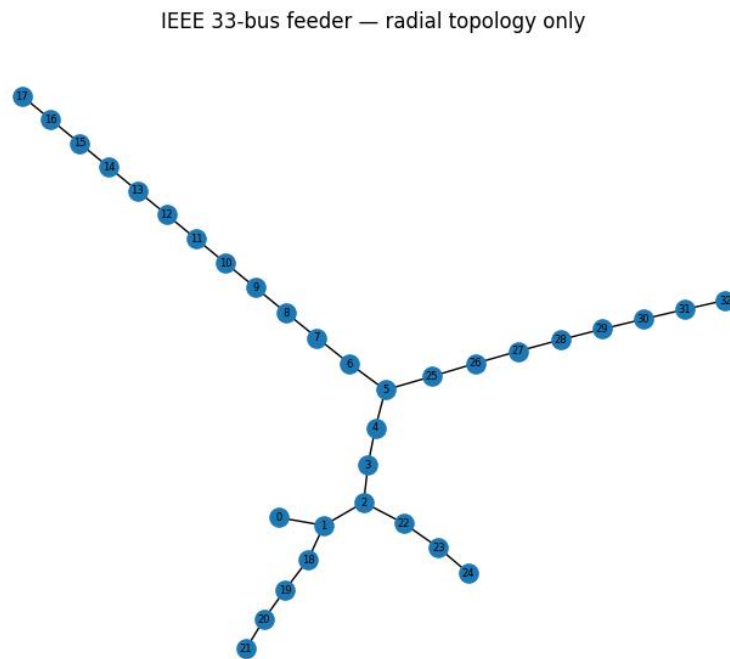


Figure 1: IEEE 33-bus feeder, radial topology.

5.2 Y_{bus} Check

With $V_{base} = 12.66$ kV and $S_{base} = 10$ MVA, the hand-built Y_{bus} reproduced **pandapower**’s own power injections to within 7.7×10^{-8} MW and 9.8×10^{-8} MVar – essentially exact agreement.

5.3 State Estimation

Table 2 compares the two methods on the test set. WLS is much more accurate than GraphSAGE.

Table 2: State estimation accuracy, test set.

Method	Vm MAE (p.u.)	Va MAE (deg)	Evaluated on
Classical WLS	0.00023	0.01511	200 samples
GraphSAGE	0.01271	0.14554	600 samples

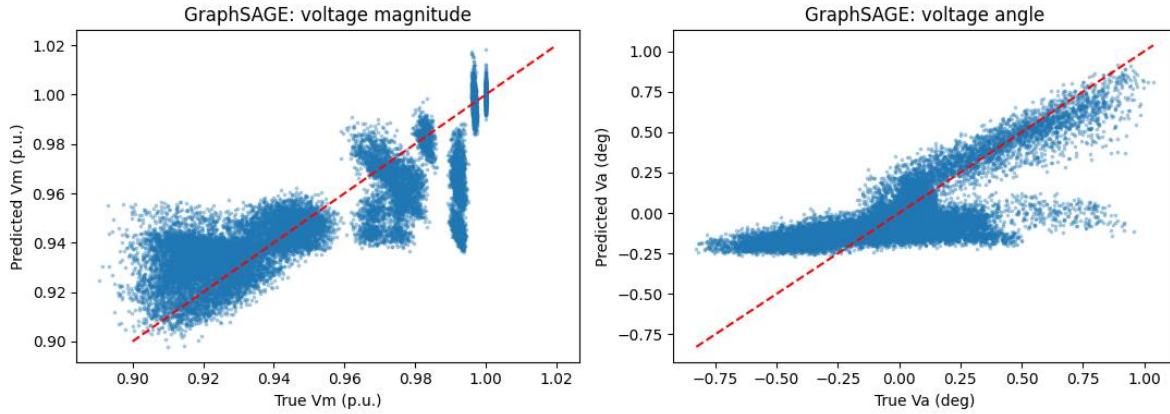


Figure 2: GraphSAGE predicted vs. true state.

5.4 Fault Diagnosis

The classifier reached 100% precision, recall, and F1-score across all five fault classes on 750 test samples (Figure 3).

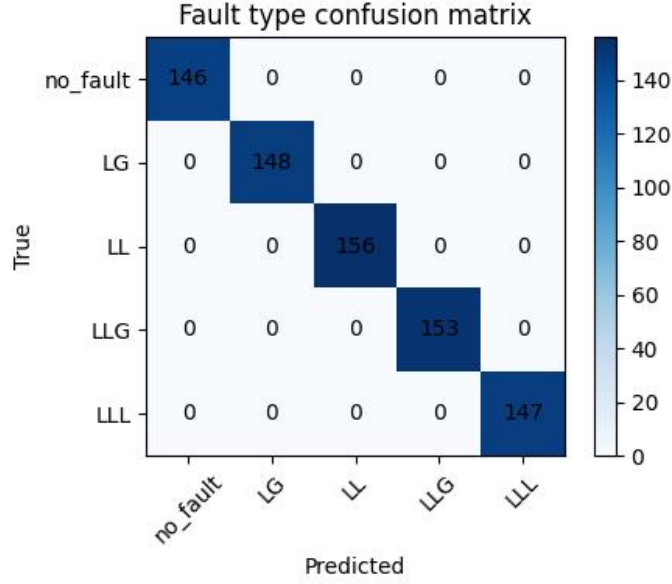


Figure 3: Fault-type confusion matrix.

5.5 FDI Detection

The residual detector achieved an AUC of 0.524, barely better than random guessing (Figure 4). A Random Forest classifier trained on residual features reached only 51% accuracy on a balanced test set. The meter-blind predictor's own test error (standardized MSE 0.5275) was about 36% higher than the main estimator's (0.3869) – this gap is the key to explaining the result below.

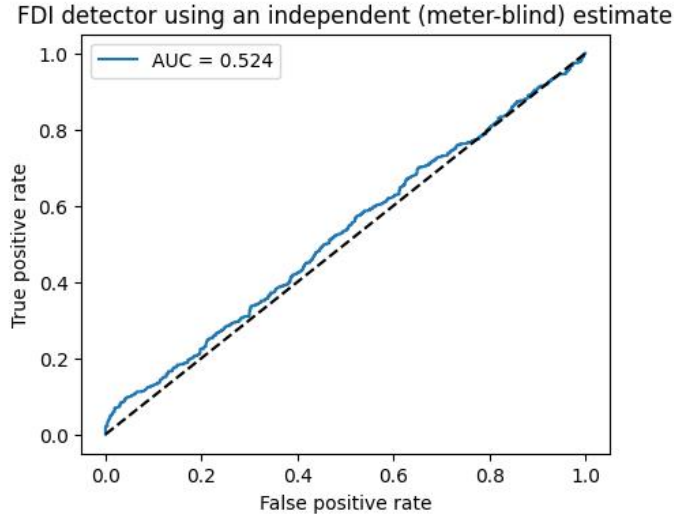


Figure 4: ROC curve, meter-blind residual detector.

6 Discussion

Why WLS beat GraphSAGE. WLS has direct access to the true nonlinear measurement equations and refines its estimate until convergence. GraphSAGE has to learn an approximation of the same physics from examples alone, using a small network trained for a fixed number of epochs. This is a fair result for a first learned model compared against a mature classical method – and it is exactly the kind of comparison a proper baseline is supposed to provide.

Why fault classification is still 100%. This time the fault labels come from real sequence-network formulas, not made-up numbers. The negative-sequence signature for a given fault type is mostly determined by the fault type itself, so the classes stay well separated even with randomized fault location and impedance. High accuracy using sequence-component features is realistic and matches published results; a real feeder would likely add some overlap through measurement noise and other effects not modeled here.

Why the FDI detector still fails. The earlier bug (the detector seeing its own answer) is fixed: the meter-blind model never receives any voltage reading, by construction. But detection is still close to random. The reason is different this time: the meter-blind model’s own prediction error is comparable to, or larger than, the attack size used in this test. A detector can only catch an attack if the attack is bigger than the detector’s own uncertainty, and here it mostly is not. This points to two concrete fixes: normalize the residual by each bus’s expected uncertainty instead of using a raw threshold, and test detection across a range of attack sizes instead of reporting a single number.

7 Conclusion

This project combined classical and graph-based learning methods for distribution monitoring, with an emphasis on reporting results honestly. The corrected network topology, the validated Y_{bus} model, the physics-based fault features, and the fixed (but still imperfect) attack detector together form a small but defensible body of work. The most useful result is not the strong WLS number or the clean fault classifier, but the FDI finding: fixing one bug revealed a different, more fundamental limitation, and telling the two apart is the main skill this project was meant to build.